

RISC-V

Exercise 1:

1. For the following C statement, write the corresponding RISC-V assembly code. Assume that the C variables f, g, and h, have already been placed in registers x5, x6, and x7 respectively. Use a minimal number of RISC-V assembly instructions.

$f = g + (h - 5);$

solution:

```
addi x5, x7, -5
add x5, x6, x5
```

2. For the following C statement, write the corresponding RISC-V assembly code. Assume that the variables i and j are assigned to registers x28 and x29, respectively. Assume that the base address of the arrays A and B are in registers x10 and x11, respectively.

$B[8] = A[i-j];$

```
sub x30, x28, x29 //compute i-j
slli x30, x30, 2 // multiply by 4 to convert the word offset to a byte offset
add x30, x10, x30 // compute the &A[i-j]
lw x30, 0(x30) // load A[i-j]
sw x30, 32(x11)
```

3. Translate the following C code to RISC-V. Assume that the variables i and j are assigned to registers x28 and x29, respectively. Assume that the base address of the arrays A and B are in registers x10 and x11, respectively. Assume that the elements of the arrays A and B are **8-byte** words:

$B[8] = A[i] + A[j];$

```
slli x28, x28, 3 // x28 = i × 8
add x28, x28, x10 // x28 = &A[i]
ld x28, 0(x28) // x28 = A[i]
slli x29, x29, 3 // x29 = j × 8
add x29, x29, x11 // x29 = &A[j]
ld x29, 0(x29) // x29 = A[j]
add x30, x28, x29 // x30 = A[i] + A[j]
sd x30, 64(x11) // B[8] = A[i] + A[j]
```

Exercise 2:

Translate to RISC-V:

1.

```
// s0 -> a, s1 -> b
// s2 -> c, s3 -> z
int a = 4, b = 5, c = 6, z;
z = a + b + c + 10;
```

```

addi s0, x0, 4
addi s1, x0, 5
addi s2, x0, 6
add s3, s0, s1
add s3, s3, s2
addi s3, s3, 10

```

2.

```
// s0 -> int * p = intArr;
// s1 -> a;
*p = 0;
int a = 2;
p[1] = p[a] = a;
```

```

sw x0, 0(s0)
addi s1, x0, 2
sw s1, 4(s0)
slli t0, s1, 2
add t0, t0, s0
sw s1, 0(t0)

```

Exercise 3:

For the following C statement, write a minimal sequence of RISC-V assembly instructions that performs the identical operation. Assume x6 =A, and x17 is the base address of C.

```
A = C [0] << 4;
```

```

lw x6 , 0( x17)
slli x6 , x6 , 4

```

Exercise 4:

1. Convert the following high-level code snippet into RISC-V assembly. Assume that the base addresses of array1 and array2 are held in t1 and t2 and that the array2 array is initialized before it is used. Use as few instructions as possible. Clearly comment your code.

```

int i ;
int array1 [100];
int array2 [100];
...
for ( i = 0; i < 100; i++)
    array1 [ i ] = array2 [ i ];

```

```

addi t0 , X0 , 0 # i = 0
addi t3 , X0 , 100 # t3 = 100
for: bge t0 , t3 , done
    slli t4 , t0 , 2      # i*= 4
    add t5 , t1 , t4      # t5 = address of array1 [i]
    add t4 , t2 , t4      # t4 = address of array2 [i]
    lw t6 , 0( t4)        # t6 = array2 [i]
    sw t6 , 0( t5) # array [i] = array2 [i]
    addi t0 , t0 , 1 # i += 1

```

```
    beq X0 , X0 , for
done:
```

2. Convert the following high-level code snippet into RISC-V assembly. Assume that the temp array is initialized before it is used and that t3 holds the base address of temp.

```
int i ;
int temp [100];
...
for ( i = 0; i < 100; i = i + 1)
    temp [ i ] = temp [ i ] * 128;
```

```
    addi s0 , X0 , 0  # i = 0
    addi t0 , X0 , 100 # t0 = 100
for: bge s0 , t0 , done
    slli t2 , s0 , 2
    add t2 , t2 , t3
    lw t4 , 0( t2)
    slli t4 , t4 , 7
    sw t4 , 0( t2)
    addi s0 , s0 , 1
    beq X0 , X0 , for
done:
```

Exercise 5:

Provide the instruction type, assembly language instruction, and binary representation of instruction described by the following RISC-V fields:

1. opcode=0x33, funct3=0x0, funct7=0x20, rs2=5, rs1=7, rd=6
2. opcode=0x3, funct3=0x3, rs1=27, rd=3, imm=0x44

1. opcode = 0x33 = 0110011 (7 bits) $\Rightarrow$ R-format (cop rd, rs₁, rs₂)
 F3 = 000 (3 bits)
 F7 = 0x20 = 0100000 (7 bits) $\Rightarrow$ sub instruction

• Assembly instruction: sub X₆, X₇, X₅

• binary representation:

31	F7	23	24	rs2	20	19	rs1	15	14	F3	11	rd	7	6	cop	0					
0100000							00101			00111			000			00110			0110011		
7 bits							5 bits			5 bits			3 bits			5 bits			7 bits		

• Hex: 0x40538333

• op code = 0x3 = 0000011 $\Rightarrow$ I-format $\rightarrow$ (cop rd, rs₁, imm)

F3 = 0x3 = 011 $\Rightarrow$ ld instr (Load) $\rightarrow$ ld rd, imm(rs₁)
 b h w

ld X₃, 4(X₂₇)

• binary representation:

31	imm	rs1	F3	rd	cop	0												
00000000100							11011			011			00011			0000011		

Hex: 0x004DB183

Exercise 6 :

What are the instructions encoded by:

- 0x003122B3
- 0x00820783
- 0x00219223

• 0x003122B3

0	0	3	1	2	2	B	3
0000	0000	0011	0001	0010	0010	1011	0011
F7		rs2	rs1	F3	rd		cop = 0x33
0		X ₃	X ₂	slt	X ₅		$\downarrow$ R-format

$\Rightarrow$ slt X₅, X₂, X₃ (set if less than)
 (if X₂ < X₃ $\Rightarrow$ rd = 1 (set)
 else X₂ > X₃ $\Rightarrow$ rd = 0 (reset))

• 0x 00 82 07 83

0000 0000 1000 | 0010 0000 | 0111 1000 0011
 imm 8 rs1 X₄ F3 1b rd X₁₅ cop = 0x3

↓
 load (I-format)

⇒ 1b X₁₅, 8 (X₄)

• 0x 00 21 92 23

0000 0000 | 0010 0001 | 1001 0010 0010 0011
 imm higher [11:5] rs2 X₂ rs1 X₃ F3 sh imm Lower [4:0] cop = 0x23

↓
 S-format

Sh X₂, 4 (X₃)

↑
 imm store
 12 bits

imm higher imm Lower
 0000000000100
 ↑ ↑
 11 0
 12 bits